

技术交底书

一、发明名称

一种面向标准机械臂批量逆运动学的CUDA并行映射与寄存器级小矩阵求解方法

二、技术领域

本发明属于机器人运动学与GPU并行计算交叉领域，具体涉及一种基于CUDA架构的标准机械臂批量逆运动学（Inverse Kinematics, IK）并行加速求解方法。该方法针对6自由度（如Universal Robots UR10）及7自由度（如Franka Panda）串联机械臂的大批量独立目标位姿逆运动学查询任务，通过CUDA Block/Warp/Lane三级并行映射、寄存器级小矩阵LDL^T分解求解、PaddedMat6x8共享内存布局优化和FP32/FP64混合精度计算路径设计，在单次GPU kernel launch中完成全部目标的阻尼最小二乘（Damped Least Squares, DLS）迭代求解，显著提升了批量IK的吞吐量、数值稳定性和性能可预测性。

三、背景技术

3.1 机械臂逆运动学求解的技术背景

逆运动学（IK）是机器人学中的基础计算原语——给定末端执行器的目标位姿（位置和姿态），求解使末端达到该位姿的关节角配置。IK广泛应用于机器人离线采样、抓取候选生成、运动规划初始化、标定辅助和轨迹生成等任务，是机器人操作流程中最频繁调用的计算模块之一。

对于6自由度全旋转关节串联机械臂，其关节变量为 $q = [q_1, q_2, q_3, q_4, q_5, q_6]^T \in \mathbb{R}^6$ 。末端执行器在基座坐标系下的位姿由齐次变换矩阵 $T_{EE}(q) \in SE(3)$ 表示，该矩阵通过6个关节变换矩阵的链式乘积得到。IK问题即：给定目标位姿 $T^d \in SE(3)$ ，求解 q^* 使得 $T_{EE}(q^*) = T^d$ 在给定容差内成立。

3.2 现有IK求解方法及其局限性

3.2.1 阻尼最小二乘法（DLS）与Levenberg-Marquardt变体

DLS是机械臂IK中最常用的迭代数值方法之一。其核心思想是将非线性IK问题在每次迭代中线性化为6×6正规方程，并通过阻尼项保证数值稳定性。LM方法在DLS基础上引入自适应阻尼调度——误差下降时减小阻尼（趋向Gauss-Newton步），误差上升时增大阻尼（趋向梯度下降步）。

DLS/LM的优势在于问题形式清楚、可直接实现、每步计算代价可分析。但其根本局限在于：当目标数量增至数百或数千时，CPU端的逐目标串行迭代导致总耗时线性增长。对于N=5000的批量IK查询，即使在单目标平均仅需1ms收敛的情况下，串行处理的总耗时也超过5秒，无法满足交互式或近实时应用的需求。

3.2.2 GPU加速机器人求解器（cuRobo / PyRoki）

cuRobo（NVIDIA Research）是目前最具代表性的GPU加速机器人运动生成系统。其核心策略是：为每个IK目标生成多个粒子（particles，默认约200个），每个粒子从不同初始种子出发，通过L-BFGS优化器并行求解，最后从多个解中选取最优者。cuRobo在较大批量（N>1000）下，通过多粒子并行有效填充GPU计算单元，实现了高SM利用率。

PyRoki（UC Berkeley）基于JAX框架实现GPU加速的运动学优化工具链，强调模块化、可组合性和研究灵活性，支持多种优化器和运动学约束的即插即用。

两类系统的共同特征是：GPU IK的性能不仅取决于数学算法本身，还取决于框架层级、kernel launch次数、批处理策略和精度路径。但它们在从小至中批量（ $N=100-1000$ ）场景下的per-target固定开销较高——cuRobo的多粒子搜索（每目标200粒子×多seed）在小批量下，每目标搜索开销占总耗时比例较高（实测 $N=100$ 时Python/C++软件栈固定开销占总耗时90%以上），未能随batch size减小而等比降低。此外，基于PyTorch等高层框架的GPU求解器在特定批量规模下可能因GPU内存管理机制（如CUDA Caching Allocator的碎片化/重分配）触发求解时间的非线性退化——实测中cuRobo在 $N=4000$ 、 7000 、 9000 、 10000 四个批量规模上出现约230ms的退化模式（约为正常32ms的7倍），且该退化与 N 值无单调关系，呈现“正常→退化→恢复→再退化”的批量振荡。

3.2.3 小矩阵线性系统的GPU求解挑战

对于6自由度机械臂，每次DLS迭代需求解的线性系统 $H \cdot \Delta q = g$ 的规模仅为 6×6 。通用线性代数库（如cuBLAS）的Warp级分块策略针对 $n \geq 32$ 的矩阵优化，其最小优化规模远大于 6×6 。对 6×6 极小矩阵而言，每次GEMM/TRSM调用需独立kernel launch，其启动和调度开销（通常为 $5-10\mu s$ 量级）可能远超实际分解计算量（ 6×6 LDLT仅需约86次标量运算，约 $0.1\mu s$ 即可完成）。此外， 6×6 矩阵在cuBLAS中无法利用Tensor Core或高效的Warp级tiling策略。

因此， 6×6 小矩阵的GPU高效求解不能直接依赖通用线性代数库，而需要专门设计的寄存器级轻量实现。

3.2.4 收敛阈值对算法评估的决定性影响

在批量IK求解器的性能评估中，收敛阈值的选择直接影响优化项的性能归因。实验表明，在宽松阈值（位置30mm/姿态 10° ）下，固定阻尼系数 λ （无自适应策略）的DLS基线配置在 $N=100$ 时吞吐达134k targets/s、收敛率1.000、平均迭代仅4.3次——表现与自适应阻尼配置接近，甚至因无分支开销而更快。然而当阈值收紧至工程合理水平（位置10mm/姿态 5° ）后，同一固定 λ 配置的收敛率骤降至0.522-0.830，平均迭代次数激增至31-80次，吞吐暴跌至7.6k-12.5k targets/s。这揭示了一个方法论问题：过于宽松的阈值会系统性地低估收敛策略的价值，甚至产生“自适应阻尼带来负收益”的伪象。

3.2.5 现有技术的综合不足

综合以上分析，现有技术在面向标准机械臂批量IK的GPU加速方面存在以下五方面不足：

- （1）**小矩阵库调用开销过大**： 6×6 线性系统的通用库求解存在不可接受的kernel launch与计算量之比；
- （2）**小批量场景效率低下**：现有GPU求解器的per-target固定开销（多粒子、多kernel launch）在 $N < 1000$ 时导致GPU计算单元利用率不足；
- （3）**迭代控制流映射困难**：DLS的自适应阻尼、收敛判定、步长控制和分支对齐等串行逻辑难以自然映射到GPU的SIMT（单指令多线程）执行模型；
- （4）**精度配置不灵活**：纯FP64实现无法充分利用消费级GPU（如Ada Lovelace架构）的高FP32:FP64吞吐比（64:1），而全FP32实现又可能损害收敛稳定性；
- （5）**批量扩展确定性不足**：基于高层框架的GPU求解器的性能在特定批量规模下不可预测——退化模式不由用户可控参数（如 N 值大小）决定，而由GPU内存管理器的内部状态决定，对需要稳定延迟的机器人控制回路构成实质性风险。

四、发明内容

4.1 本发明要解决的技术问题

针对上述现有技术的不足，本发明要解决的技术问题是：

问题一（并行映射问题）：如何将批量IK的DLS迭代过程——包括FK、数值Jacobian、阻尼正规矩阵构造、LDLT求解和关节更新——高效映射到CUDA的Block/Warp/Lane三级并行层次，使得N个独立目标的全部迭代在单次kernel launch中完成，消除per-target和per-iteration的launch累积开销。

问题二（小矩阵求解问题）：如何在GPU上以最低开销求解每次DLS迭代中的 6×6 对称正定线性系统，避免通用线性代数库的kernel launch和全局内存访问开销。

问题三（内存布局问题）：如何消除共享内存中 6×6 矩阵列访问导致的Bank冲突——该冲突在 $H = J^T \cdot W^2 \cdot J$ 构造阶段将共享内存有效吞吐降低至理论峰值的约1/3。

问题四（精度效率平衡问题）：如何在消费级GPU上平衡FP32的高吞吐与FP64的高精度，使批量IK求解在维持99.8%以上收敛率的同时，吞吐量提升120-149%。

问题五（批量扩展确定性问题）：如何使批量IK求解器在任意批量规模下的性能行为完全可预测——GPU时间与N严格线性、吞吐恒定，消除因GPU内存管理机制或框架开销导致的非线性退化。

4.2 技术方案

本发明提出一种面向标准机械臂批量逆运动学的CUDA并行映射与寄存器级小矩阵求解方法，包括以下四项核心技术方案。

4.2.1 发明点一：Block级批量并行映射与Warp级任务分工方法

核心思想：将每个IK目标映射为1个CUDA block（含128个线程=4个Warp $\times$ 32个Lane），block内按Warp分工协作完成DLS迭代的全部计算步骤，全部DLS迭代循环（最大 $K_{\max}=160$ 次迭代）在单次kernel launch中完成，所有N个目标并行求解。

（a）Grid到Block的映射

设批量规模为N（目标位姿数量），目标位姿矩阵为 $T_{\text{tgt}}^{(b)} \in SE(3)$ ($b = 0, 1, \dots, N-1$)，关节角种子为 $q^{(b,0)} \in \mathbb{R}^6$ 。Grid维度定义为：

$\text{Grid} = (N, 1, 1)$

即每个目标位姿分配一个CUDA block，各block之间完全独立——无需跨block通信，无需全局同步。Block索引 $b = \text{blockIdx.x}$ 直接标识目标ID。

（b）Block内部128线程4-Warp分工

每个Block固定为128线程（4 Warps $\times$ 32 Lanes/Warp）。该配置通过约束驱动的排除法从三种候选方案中选出：

- **候选方案A（每线程一任务，Fine-grained）：**每个线程独立处理一个完整目标。寄存器需求估算约372个/线程（J: 64 + H: 64 + 6段FK中间 4×4 矩阵: 192 + 误差/梯度/步长: 36 = 372个32位寄存器），远超硬件上限（255个/线程），导致local memory spilling，吞吐急剧下降。排除。
- **候选方案B（每Block一任务，Cooperative）：**1 block/target，block内线程协作。寄存器需求约94-98个/线程，共享内存需求约1,616字节/block。Nsight Compute实测：94寄存器/线程（B4 FP64），98寄存器/线程（B5混合精度），1.616 KB共享内存/block，无local memory spill。采用。

- **候选方案C（每Warp一任务）**：1 warp/target，32线程/warp。Warp内线程数不足以并行展开Jacobian 6列（需6路数据并行的FK计算），且4个warp/block可恰好对应4项主要任务类别。仅当DOF≥12且J矩阵规模超过warp容量时考虑。排除。

最终映射方案为128线程/Block（4 Warps），Warp级任务分工如下表所示：

Warp	Lane 范围	计算任务	数学映射	并行粒度	执行模式
W0	0--31	FK链式乘法、位姿误差计算、LDLT求解、LM阻尼更新、收敛判定	全部DLS控制流	串行控制	Lane 0串行执行
W1	32--63	数值雅可比矩阵6列并行组装	J的第j列由Lane 32+j负责，每列2次FK扰动计算	6路数据并行	数据并行
W2	64--95	阻尼正规矩阵H与梯度g构造	Lane 64+(i×6+j)负责H_{ij}元素的内积计算	36路数据并行	数据并行
W3	96--127	关节角步长更新、限位投影	6路数据并行	6路数据并行	数据并行

(c) Warp内线程任务的数学映射

Warp 0（Lane 0-31，控制流）：Lane 0执行全部控制流逻辑，其余31个Lane处于等待状态。Warp 0执行的主要操作包括：

- 前向运动学（FK）：根据当前关节角q计算末端位姿T(q)，读取 `__constant__` 内存中的运动学参数（`c_segment_origins[96]`、`c_segment_axes[18]`、`c_q_index[6]`、`c_T_wrist3_to_tcp[16]`），调用Rodrigues旋转公式和4×4齐次矩阵乘法链式计算。单次FK运算量约为~450次乘法+12次三角函数（6次Rodrigues + 5次4×4矩阵乘法 + 1次工具变换）。
- 位姿误差计算：位置误差 $e_p = p^d - p(q) \in R^3$ ；姿态误差采用李代数形式 $e_R = \text{Log}(R^d \cdot R(q)^T) \vee \in R^3$ ；总误差 $e = [e_p; e_R] \in R^6$ 。
- LDLT求解：将共享内存中的H矩阵（PaddedMat6x8布局）拷贝至寄存器中的6×6密集数组，调用寄存器级`ldlt_solve_6x6()`函数完成分解和前代/回代求解。
- LM阻尼更新：根据收敛状态（误差改进/退化/停滞）调整阻尼参数λ。
- 收敛判定：采用三档递进阈值体系——Loose ($\epsilon_p=0.03\text{ m}$, $\epsilon_R=0.1745\text{ rad}$, 30mm/10°)、Medium ($\epsilon_p=0.01\text{ m}$, $\epsilon_R=0.0873\text{ rad}$, 10mm/5°, 主 benchmark) 和 Strict ($\epsilon_p=0.005\text{ m}$, $\epsilon_R=0.0175\text{ rad}$, 5mm/1°)——位置误差与姿态误差独立判定，两者同时满足各自阈值方标记为收敛。

Warp 1（Lane 32-63，6路数据并行）：Lane 32+j（j=0,...,5）负责计算Jacobian的第j列。采用中心差分法：

$$J_{\{:,j\}} = [e(q+\epsilon \cdot e_j) - e(q-\epsilon \cdot e_j)] / (2\epsilon)$$

其中 $\epsilon = 1 \times 10^{-6}$ 为扰动步长， e_j 为单位向量。每列需2次FK扰动计算——每个Lane独立执行：

1. 从共享内存拷贝当前q至本地寄存器
2. 对第j个关节施加正/负扰动：`q_plus[j] += ε`, `q_minus[j] -= ε`
3. 分别调用`forward_kinematics()`计算T_plus和T_minus

4. 计算位姿误差 e_{plus} 和 e_{minus}
5. 中心差分： $J_{\{i,j\}} = (e_{plus} - e_{minus}) / (2\epsilon)$

12次FK计算（6列 $\times$ 2次扰动）被压缩为约2次FK延迟，因为6个Lane的FK计算在SM内并行执行。

Warp 2 (Lane 64-95, 36路数据并行)： Lane $64+(i \times 6+j)$ ($i,j=0,\dots,5$) 负责计算 $H_{\{ij\}}$ 元素。每个Lane执行一个6项加权内积：

$$H_{\{ij\}} = \sum_{k=0}^5 w_k^2 \cdot J_{\{ki\}} \cdot J_{\{kj\}}$$

其中 w_k 为权重矩阵 W 的对角元素。对角线元素加阻尼： $H_{\{ii\}} += \lambda^2$ 。Lane还负责计算梯度向量的对应分量： $g_i = \sum_{k=0}^5 w_k^2 \cdot J_{\{ki\}} \cdot e_k$ 。

该阶段的关键特征是：36个Lane同时从共享内存读取Jacobian矩阵的列——若无PaddedMat6x8布局优化，stride=6的自然行主序布局将导致2-3路Bank冲突（详见发明点三）。

Warp 3 (Lane 96-127, 6路数据并行)： Lane $96+i$ ($i=0,\dots,5$) 负责第 i 个关节的步长更新和限位投影：

- 关节限位投影： $q_{i^{(k+1)}} = \text{clamp}(q_{i^{(k)}} + \Delta q_i, q_{\{i,\min\}}, q_{\{i,\max\}})$
- 步长由 $\|\Delta q\|_\infty \leq 0.25 \text{ rad}$ 的硬约束确定：若 $\|\Delta q\|_\infty \leq 0.25$ 则 $\alpha=1.0$ （全步长）；否则 $\alpha=0.25/\|\Delta q\|_\infty$ （等比缩放）

(d) 单次Kernel Launch完成全部DLS迭代

区别于现有GPU求解器（如cuRobo）的多次kernel launch策略（per-target或per-iteration），本发明将整个DLS迭代循环（最大 $K_{\max}=160$ 次迭代）封装在单个kernel内部。N个block各自独立运行至多 $K_{\max}$ 次迭代，无需CPU端的中途干预和重新launch。单kernel架构的关键优势包括：

1. **消除kernel launch累积开销：** cuRobo等框架型求解器的跨语言调用栈（Python $\rightarrow$ C++ $\rightarrow$ CUDA $\rightarrow$ C++ $\rightarrow$ Python）在 $N=100$ 时固定开销占总耗时90%以上。
2. **迭代增量成本线性可控：** 每增加一轮迭代的边际成本仅约0.02 ms ($N=5000$ 条件下)，不存在host-device同步的放大效应。
3. **GPU时间与N严格线性：** 实测 $R^2 > 0.999$ ——N翻倍意味着block数翻倍，在SM数量固定的条件下GPU时间精确翻倍，吞吐保持恒定。
4. **完全确定性的性能行为：** 不存在动态内存分配、无跨block通信、无GPU-CPU同步——吞吐在任何N值下均可从事先的计算精确预测。

4.2.2 发明点二：寄存器驻留的 6×6 LDLT分解求解器

核心思想： 利用 6×6 矩阵规模固定且足够小的特性，将完整的LDLT分解和三角求解过程全部置于GPU寄存器中执行，消除通用线性代数库的kernel launch开销、全局内存往返和共享内存访问。

(a) LDLT分解的数学形式

对于 6×6 对称正定矩阵 $H = J^T \cdot W^2 \cdot J + \lambda^2 I$ ，LDLT分解（无需选主元）将 H 分解为：

$$H = L \cdot D \cdot L^T$$

其中 L 为单位下三角矩阵（ $L_{\{ii\}}=1$ ）， D 为对角矩阵。该分解的选择基于以下考虑：

- 与Cholesky分解（ $H = L \cdot L^T$ ）相比，LDLT避免了6次平方根运算——GPU上平方根指令的吞吐量约为FMA（融合乘加）的1/4至1/8。

- $\lambda^2 I$ 正则项保证 H 的对称正定性，因此无需选主元（无需行交换），分解过程数值稳定。

(b) LDL^T 分解算法（寄存器内实现）

算法步骤（所有中间变量均为寄存器内标量或小数组）：

阶段一： LDL^T 分解

```
for j = 0 to 5:
    // 对角元更新
    D[j] = A[j][j]
    for k = 0 to j-1:
        D[j] -= L[j][k] * L[j][k] * D[k]

    L[j][j] = 1.0

    // 非对角元更新
    for i = j+1 to 5:
        L[i][j] = A[i][j]
        for k = 0 to j-1:
            L[i][j] -= L[i][k] * L[j][k] * D[k]
        L[i][j] /= D[j]
```

阶段二：前代求解 ($L \cdot y = g$)

```
for i = 0 to 5:
    y[i] = g[i]
    for k = 0 to i-1:
        y[i] -= L[i][k] * y[k]
```

阶段三：对角缩放 ($D \cdot z = y$)

```
for i = 0 to 5:
    y[i] /= D[i]
```

阶段四：回代求解 ($L^T \cdot \Delta q = z$)

```
for i = 5 down to 0:
    dq[i] = y[i]
    for j = i+1 to 5:
        dq[i] -= L[j][i] * dq[j]
```

(c) 运算量统计

阶段	循环范围	标量运算次数	可FMA映射
对角元更新	$j=0..5, k=0..j-1$	15	15
非对角元更新	$j=0..5, i=j+1..5, k=0..j-1$	20	20
L缩放（除D _j ）	$j=0..5, i=j+1..5$	15	0（除法）
前代求解	$i=0..5, k=0..i-1$	15	15
对角缩放	$i=0..5$	6	0（除法）
回代求解	$i=5..0, j=i+1..5$	15	15
合计		86	65

总计86次标量运算中，65次为乘加/乘减运算（可映射为单周期FMA指令），21次为除法（由SFU特殊功能单元流水线执行）。总耗时约0.1μs，完全在寄存器中完成，零内存流量。每轮迭代时间在1.945-1.951 μs内波动（<0.3%）——无论阻尼因子λ如何变化，LDL^T的计算路径和操作数均不变，不存在LM线搜索的步长回退风险。

(d) 寄存器驻留分析

LDL^T求解阶段涉及的完整数据集包括：

- H矩阵（36个FP64元素）+ L矩阵（36个FP64元素）+ D对角元（6个FP64元素）+ 中间向量y, z, dq（18个FP64元素）= 96个FP64元素 = 768字节 = 约192个32位寄存器的等效存储空间。

在Ada Lovelace架构（255寄存器/线程上限，128线程/block）下，上述数据集完全可驻留在单个Lane的寄存器文件中，无需local memory spilling（Nsight Compute实测确认：local memory spill = 0）。

(e) 编译时优化

矩阵维度6为编译时常量（constexpr），三重循环的迭代次数（ $j=0..5, k=0..j-1, i=j+1..5$ ）完全确定。通过NVCC的 `#pragma unroll` 指令，编译器将全部循环展开为直线代码序列：

- 消除分支预测失败（无条件跳转）
- 消除循环控制开销（索引递增和条件检查）
- 编译器可进一步应用指令级并行优化（ILP），将无数据依赖的标量运算调度至同一指令发射周期

(f) 与通用库方案的技术效果对比

对比通用线性代数库（cuBLAS GEMM/TRSM）：通用库求解极小矩阵时，kernel launch与全局内存往返等开销远超实际分解计算量（86次标量运算 vs 5-10μs launch开销）；本发明寄存器LDL^T求解阶段本身不产生全局内存流量——这对DRAM带宽受限的批量IK场景具有关键意义。

(g) FP64精度的数值稳定性作用

FP64提供15位有效数字，即使在Jacobian条件数达10⁶时仍能保证分解精度。DLS求解器在14-16轮迭代后的自然收敛精度（Strict阈值下PosErr均值1.8 mm, RotErr均值远低于1°）已大幅超越最严格的阈值要求——收敛不是因为“恰好满足阈值”，而是因为求解器本身的精度上限足够高。这一属性是本发明三档阈值鲁棒性的核心结构基础之一。

4.2.3 发明点三：PaddedMat6x8共享内存布局的Bank冲突消除方法

核心思想：将共享内存中6×6矩阵的行步长从自然值6扩展为8（添加2列零填充），通过改变Bank访问模式消除stride=6布局导致的系统性Bank冲突。

(a) Bank冲突的成因分析

CUDA共享内存由32个Bank组成，每个Bank宽度为4字节。连续的4字节字映射到连续的Bank（字地址w映射到Bank $w \bmod 32$ ）。当同一Warp内的多个线程同时访问同一Bank的不同地址时，发生Bank冲突——访问被串行化，有效带宽降低。

在6×6矩阵的自然行主序布局（stride=6 doubles = 48 bytes = 12 banks per row）中：

- 地址映射： $\text{addr}(r,c) = r \times 12 + c \times 2$ （每个double占2个4字节字）
- Bank映射： $B(r,c) = (r \times 12 + c \times 2) \bmod 32$

在 $H = J^T \cdot W^2 \cdot J$ 构造阶段（Warp 2的36线程并行），线程(i,j)需读取Jacobian的第i行和第j行进行列内积计算。以读取J的第c列为例，6个元素访问的Bank序列为：

$\{2c, 12+2c, 24+2c, 36+2c \equiv 4+2c, 48+2c \equiv 16+2c, 60+2c \equiv 28+2c\} \pmod{32}$

由于 $\text{gcd}(12, 32)=4$ ，Bank访问模式以 $32/4=8$ 为周期重复。在6行访问中，不可避免地发生Bank索引的模冲突，导致2-3路Bank冲突。Nsight Compute实测确认该冲突模式：共享内存吞吐量仅为理论峰值的约1/3。

(b) PaddedMat6x8布局方案

将6×6矩阵的行步长扩展至8个double（即6×8布局），添加2列零填充。地址映射变为：

$\text{addr}(r,c) = r \times 16 + c \times 2$

Bank映射： $B'(r,c) = (r \times 16 + c \times 2) \bmod 32$

关键性质：由于 $\text{gcd}(16, 32)=16$ ，Bank访问模式以 $32/16=2$ 为周期。

- 偶数行（ $r = 0, 2, 4$ ）：起始Bank索引为 $c \times 2$ （Bank 0-15范围）
- 奇数行（ $r = 1, 3, 5$ ）：起始Bank索引为 $16 + c \times 2$ （Bank 16-31范围）

两组使用的Bank集合完全不重叠。在LDLT分解和阻尼正规矩阵构造等核心计算中——这些操作每次仅涉及 ≤ 2 行的同时访问——因此可消除stride=6布局导致的主要系统性Bank冲突。

(c) PaddedMat6x8的C++零开销封装

```
struct PaddedMat6x8 {
    double* data; // 指向 __shared__ double[6×PADDEDMAT_STRIDE]

    __device__ __forceinline__ double& operator()(int row, int col) {
        return data[row * PADDEDMAT_STRIDE + col];
    }
    __device__ __forceinline__ const double& operator()(int row, int col)
const {
    return data[row * PADDEDMAT_STRIDE + col];
}
};
```


其中PADDED_MAT_STRIDE为编译时常量（消融级别 $\geq B2$ 时为8，否则为6）。`__device__`
`__forceinline__` 确保编译后为单条索引计算指令，NVCC -O2（sm_89）编译后PTX指令序列与裸指针访问一致——无额外抽象性能开销。

(d) 存储开销分析

额外存储 = 6行 $\times$ (8-6)列 $\times$ 8 bytes/double = 96 bytes/矩阵。对于Jacobian和Hessian两个矩阵，总额外存储 = 192 bytes，占SM共享内存总量（100 KB）的约0.19%。该代价可忽略不计。

(e) 实测验证

Nsight Compute实测数据确认：

- B4配置（FP64 + PaddedMat6x8）：shared bank conflicts = 3,522次
- B5配置（PaddedMat6x8 + FP32数据宽度）：shared bank conflicts = 1,295次，较B4减少49%。

B5的bank conflict进一步降低源于FP32数据宽度为FP64的一半（4 bytes vs 8 bytes），对共享内存的自然对齐访问减少了两路和三路bank冲突的发生频率。但即使以B4的峰值3,522次冲突计，其占总共享访问波前的比例不足2%，bank conflict的减少是混合精度加速的伴随现象而非主因。

4.2.4 发明点四：FP32/FP64混合精度计算路径

核心思想：将批量IK的DLS计算管线按精度敏感度划分为FP32段和FP64段——FK、Jacobian、H和g构造等计算密集型操作在FP32下执行以获得高吞吐（利用Ada Lovelace架构64:1的FP32:FP64吞吐比）；LDLT求解、阻尼更新和收敛判定等数值敏感操作保留FP64以保证解的质量。

(a) 精度敏感度分析

完整DLS迭代管线包含两类不同精度敏感度的运算：

(a) **低精度敏感度运算：**FK链式乘法（Rodrigues旋转公式+4 $\times$ 4矩阵乘法）、位姿误差计算（向量减法+旋转矩阵乘法）和数值Jacobian组装（中心差分+线性近似）。这些运算涉及大量三角函数和矩阵乘法，其精度需求为：Jacobian扰动步长 $\epsilon=1\times 10^{-6}$ 在FP32下引入的相对舍入误差约为 1×10^{-7} 量级，远小于迭代收敛容差（ 1×10^{-2} 量级），因此FP32精度损失可忽略。

(b) **高精度敏感度运算：**LDLT线性系统求解涉及条件数敏感的除法运算（ $L_{\{ij\}} = \dots / D_j$ ）和累积求和（前代/回代中的 $\Sigma L_{\{ik\}} \cdot y_k$ ）。H和g的元素累加涉及36个内积，每个内积为6项加权乘积之和。FP32累加的截断误差传播可能影响搜索方向的质量，因此这些运算需FP64精度。

(b) 混合精度数据流设计

FP32计算段（高吞吐，~90%计算量）：

```

├─ 前向运动学(FK): forward_kinematics_f(s_q, s_T)
│   └─ 运算量: 6次Rodrigues + 5次4 $\times$ 4矩阵乘法
├─ 位姿误差计算: pose_error_f(s_T, s_T_tgt, s_err)
│   └─ 位置误差: 向量减法; 姿态误差: 旋转矩阵乘法+反对称分解
├─ 数值雅可比组装: J的6列中心差分计算 (Warp 1, 6路并行)
│   └─ 每列2次FP32 FK扰动计算

├─ FP32  $\rightarrow$  FP64 类型转换边界 ─┐
│   (Hessian和gradient以FP64累积) ┘
```

FP64累积段（高精度）：

- └ H矩阵构造： $H_{ij} = \sum_k w_k^2 \cdot J_{ki} \cdot J_{kj}$ （FP64累加）
- └ g向量构造： $g_i = \sum_k w_k^2 \cdot J_{ki} \cdot e_k$ （FP64累加）
- └ $H_{ii} += \lambda^2$ （FP64阻尼项）

└ FP64 LDL^T求解（高精度关键路径）└

FP64求解段：

- └ LDL^T分解：寄存器内FP64运算
- └ 前代+对角缩放+回代求解：全部FP64
- └ LM阻尼更新： $\lambda *= 0.7$ or 2.0 （FP64）
- └ 收敛判定： $\|e_p\|_2$ 和 θ_R 与阈值比较（FP64）

└ FP64 → FP32 类型转换边界 └
└（关节步长 Δq 以FP32写回）└

FP32写回段：

- └ 关节更新+限位投影：FP32

(c) 类型转换点实现

在LDL^T求解入口处，将FP32共享内存中的Hessian矩阵和梯度向量显式转换为FP64寄存器数组：

```
double H_dense[36], g_dense[6];
for (int r = 0; r < 6; r++) {
    for (int c = 0; c < 6; c++) {
        H_dense[r*6 + c] = (double)H(r, c); // FP32 → FP64
    }
    g_dense[r] = (double)s_g[r]; // FP32 → FP64
}
ldlt_solve_6x6(H_dense, g_dense, s_dq); // FP64 LDLT核心
```

LDL^T求解完成后，将FP64结果 Δq 以FP32精度写回共享内存用于后续关节更新。

(d) 精度验证

混合精度B5配置的收敛率实测结果：N=100为1.000（100%），N=500为0.998（99.8%），N=5000为0.9998（99.98%），N=10000为0.9996（99.96%）。所有批量下收敛率均不低于99.8%，未因主体精度的降低而产生可测量的收敛退化。这验证了FP32/FP64精度边界划分的正确性——LDL^T求解器、阻尼调节和收敛判定保留FP64精度路径足以保证整体求解质量。

从Nsight Compute指标看，混合精度引入的类型转换（FP32 → FP64于LDL^T求解入口，FP64 → FP32于结果回写）没有表现为主导瓶颈；B5主要瓶颈仍为计算吞吐（60.73%）与寄存器压力（98寄存器/线程）。

4.3 有益效果

本发明相比现有技术具有以下有益效果：

(1) 批量吞吐量显著提升，小批量优势尤为突出。 在统一benchmark条件下（同一UR10模型、同一tool0、同一目标位姿/种子、统一Medium阈值10mm/5°和时间口径），本发明B5混合精度配置与cuRobo的吞吐对比如下：

批量N	CUDA B5吞吐	cuRobo吞吐	加速比
100	112,414 t/s	3,118 t/s	36.1×
500	158,251 t/s	15,844 t/s	10.0×
1000	148,412 t/s	31,611 t/s	4.7×
5000	168,683 t/s	155,059 t/s	1.09×

在N=100条件下，本发明per-target固定开销仅约8.9 μ s（0.89 ms分摊至100目标），而cuRobo的Python/C++软件栈固定开销占总耗时90%以上。即使在FP64全精度配置（B3）下，CUDA DLS在N=100时的吞吐（51,361 targets/s）仍为cuRobo的16.5倍。

(2) 全量程批量扩展严格线性，性能完全可预测。 在N=100→10000全量程扫描（12个N值，步长1000）中，CUDA B5吞吐保持148k-174k targets/s（ $\pm 8\%$ 波动），GPU时间与N的线性拟合 $R^2 > 0.999$ ——这是1 block/target映射的教科书级线性扩展行为。与之形成鲜明对比的是，cuRobo在4/12个N值（N=4000, 7000, 9000, 10000）上触发约230ms退化模式（7×正常求解时间），呈现“正常→退化→恢复→再退化”的批量振荡——这种不可预测的性能震荡是实际工程部署中比性能差距更棘手的问题。本发明通过裸CUDA内存管理（固定2KB/target，一次性cudaMalloc）、无PyTorch依赖和单kernel全迭代封装三项设计决策，从根本上消除了此类GPU内存管理导致的非线性退化。

(3) 消融实验量化各优化的独立贡献。 B0-B6七级消融实验清晰揭示了各项技术方案的贡献（Medium阈值10mm/5°）：

消融步骤	变更内容	N=100效果	N=500效果	N=5000效果
B0→B1	常量内存广播	N=100合计<5%	N≥500可忽略	N≥500可忽略
B1→B2	PaddedMat6x8	同上	同上	同上
B2→B3	自适应阻尼	ConvRate 0.830→1.000，吞吐 +577%	ConvRate 0.522→1.000，吞吐 +530%	ConvRate 0.564→1.000，吞吐 +428%
B3→B5	混合精度 (FP32+FP64)	吞吐+120%， ConvRate 1.000	吞吐+149%， ConvRate 0.998	吞吐+149%， ConvRate 0.9998

关键发现：(a) Medium阈值暴露了固定 λ 在宽松阈值下被掩盖的脆弱性——B0的收敛率在宽松阈值下虚高为1.000，收紧后骤降至0.522-0.830；(b) 自适应阻尼是所有可量化消融项中对收敛率影响最显著的单一因素——将收敛率从43-48%未收敛恢复至100%，迭代次数从73-80次锐减至13-14次（减少59-83%）；(c) 混合精度在所有批量上提供120-149%的额外吞吐提升，且收敛率无损（ $\geq 99.8\%$ ）；(d) B1/B2（内存层次优化）合计贡献<5%，B4（步长钳位+分支对齐）在所有批量上吞吐均低于B3（-15%至-20%），B6（CUDA Graph）N=100时+3.7%——验证了本发明的设计焦点（自适应阻尼收敛策略和混合精度计算路径）与硬件瓶颈准确对齐。

(4) 计算边界属性明确，架构级优化方向清晰。 Nsight Compute profiling表明本发明kernel为计算吞吐与寄存器压力约束型（compute-bound），而非DRAM带宽约束型：

指标	B4 (FP64)	B5 (混合精度)	变化
Compute throughput	66.89%	60.73%	—
DRAM throughput	1.56%	1.16%	—
L1/TEX hit rate	99.13%	98.62%	—
Registers/thread	94	98	+4
Shared bank conflicts	3,522	1,295	-49%
Kernel duration (N=100)	2,920 μ s	827 μ s	-72%

该数据说明：DRAM带宽不是瓶颈（利用率<2%）；寄存器压力（94-98/线程）是occupancy（~33%）的主因；B5加速主要来自FP32计算延迟降低，bank conflict减少是伴随现象。

(5) 三档阈值鲁棒性——求解器性能对收敛精度不敏感。 本发明在三档递进阈值（Loose 30mm/10° → Medium 10mm/5° → Strict 5mm/1°）下，CUDA B5吞吐波动仅±6%，收敛率维持0.998以上。即使在Strict (5mm/1°)极端条件下，N=5000仍以164,030 t/s领先cuRobo（加速比1.05×）。该阈值鲁棒性来源于三项底层设计协同：(a) 单kernel全迭代封装——每增加一轮迭代的边际成本仅~0.02 ms (N=5000)，不存在host-device同步放大效应；(b) FP64 LDLT数值稳定性——15位有效数字使DLS自然收敛精度远超最严格阈值要求；(c) 寄存器LDLT固定成本——每轮86次标量运算在寄存器中完成，单轮时间波动<0.3%。平均迭代次数从Loose到Strict仅增加6.5%（13.95 → 14.86 at N=100），远低于160的最大迭代上限。

(6) 框架可扩展至不同自由度平台。 本发明方法的DOF维度为编译时参数化设计（非硬编码为6DOF）。7DOF Panda机械臂的正确性验证结果表明：CUDA FK与CPU参考在机器精度量级一致（最大逐元素误差 2.78×10^{-16} ），IK收敛率与Python CPU参考完全一致。从UR10（6DOF）到Panda（7DOF）的迁移仅需：DOF从6→7、Jacobian从6×6→6×7、阻尼正规矩阵从6×6→7×7、LDLT从6×6→7×7。Warp分工方案、线程映射和自适应阻尼控制逻辑保持不变。

五、附图说明

本技术交底书包含以下六幅附图（以文字描述形式给出，供后续绘制正式附图）：

图1：系统总体架构图。 展示从URDF模型解析→CUDA常量参数导出→目标位姿/种子生成→GPU Kernel求解→结果验证的完整数据流。包括：左侧的离线预处理阶段（URDF解析模块、常量内存头文件导出、标准资产生成），中央的GPU在线求解阶段（常量内存加载、目标/种子上传、ik_batch_solve单次kernel launch、结果回读），以及右侧的后处理验证阶段（收敛率统计、误差分析、同类求解器对比）。

图2：Block/Warp/Lane三级并行映射示意图。 展示Grid(N,1,1)到Block(128线程, b=blockIdx.x)到4个Warp（各32 Lane）的层次映射。每个Block对应一个IK目标，4个Warp按数学任务分工：W0执行FK/误差/LDLT/阻尼/收敛判定（Lane 0串行控制），W1执行数值Jacobian 6列并行（6路数据并行），W2执行阻尼正规矩阵H与梯度g构造（36路数据并行），W3执行关节更新/限位投影（6路数据并行）。

图3：PaddedMat6x8共享内存布局与Bank访问模式对比图。 左侧为stride=6（非填充）布局下的Bank映射表，标注Bank冲突发生位置（gcd(12,32)=4导致的冲突周期性）；右侧为stride=8（PaddedMat6x8）布局下的Bank映射表，标注偶数行（Bank 0-15）和奇数行（Bank 16-31）的Bank集合分离（gcd(16,32)=16）。

图4：混合精度计算路径数据流图。以数据流图形式展示FP32→FP64→FP32的精度转换边界。FP32段（绿色，~90%计算量）：FK、位姿误差、Jacobian组装；FP32→FP64转换点（橙色）；FP64累积段（蓝色）：H和g的FP64累加；FP64求解段（蓝色）：LDLT分解求解（86次标量运算，零内存流量）、阻尼更新、收敛判定；FP64→FP32转换点（橙色）；FP32写回段（绿色）：关节更新和限位投影。

图5：LDLT分解求解流程图。展示从H矩阵输入到 Δq 输出的四阶段流程：阶段一（LDLT分解）：对角元更新→非对角元更新→L缩放；阶段二（前代求解 $L \cdot y = g$ ）；阶段三（对角缩放 $D \cdot z = y$ ）；阶段四（回代求解 $L^T \cdot \Delta q = z$ ）。标注每阶段的运算次数（总计86次标量运算，65次可FMA映射）和编译时完全展开特性。

图6：B0-B6消融实验效果对比图。以分组柱状图形式展示七级消融配置在不同批量（ $N=100/500/5000$ ）下的吞吐量（targets/s，对数纵轴）和收敛率。X轴为消融级别（B0-B6），每个批量一个柱组。用颜色区分批量规模，用数据标签标注关键数值，突出自适应阻尼（B2→B3）对收敛率的决定性恢复作用和混合精度（B3→B5）的吞吐倍增效应。

六、具体实施方式

6.1 系统总体架构

本发明的一个优选实施例基于以下软硬件环境实现：

硬件平台：

- GPU: NVIDIA GeForce RTX 4070 Laptop GPU (Ada Lovelace, sm_89, 8 GB VRAM)
- CPU: 主机侧为x86-64处理器
- CUDA版本: CUDA 12.x

软件模块组成（参照 `standard_robot_cuda_ik/` 目录结构）：

（1）**URDF模型解析模块**：解析标准URDF格式的机器人描述文件，提取关节运动学参数，包括关节原点变换矩阵（96个FP64元素）、旋转轴向量（18个FP64元素）、关节索引映射（6个int）、工具坐标系变换（16个FP64元素）和关节限位（12个FP64元素）。

（2）**CUDA常量参数导出模块**：将解析后的运动学参数导出为C++头文件（如 `ur10_model_constants.h`），包含16位十六进制浮点精度常量数组，确保CPU和GPU端的bit-exact一致性。运动学参数总计1,384字节，全部通过 `__constant__` 内存广播至所有block。

（3）**标准资产生成模块**：基于固定随机种子（seed=42）和目标数量（ $N \in \{100, 500, 1000, 2000, 3000, 4000, 5000, 6000, 7000, 8000, 9000, 10000\}$ ），从home位姿出发在关节空间平滑采样生成可达目标位姿集合（位置+旋转矩阵+四元数），以及对应的初始关节角种子（zero_seed策略——全零关节角初始化）。

（4）**GPU求解器模块**：核心CUDA kernel实现，包含ik_batch_solve主kernel函数和配套的设备端工具函数（forward_kinematics、pose_error、ldlt_solve_6x6、PaddedMat6x8/FloatPaddedMat6x8封装等）。

（5）**消融编译系统**：通过 `ABLATION_LEVEL` 预处理器宏和条件编译指令，从单一代码库编译生成B0-B6七级消融可执行文件。

（6）**Benchmark运行与评测模块**：Python封装的benchmark脚本，负责调用各求解器（CUDA/cuRobo/PyRoki/KDL/numeric_dls）、采集CUDA event时间指标和收敛率、输出CSV/JSON/MD格式结果。统一实验规程：每个(N, 配置)组合执行3次预热+30次正式重复，结果取算术均值。

6.2 运动学模型建立

6.2.1 UR10正运动学模型

UR10为6自由度全旋转关节串联机械臂，其关节构型由6维关节角向量描述：

$$\mathbf{q} = [q_1, q_2, q_3, q_4, q_5, q_6]^T \in \mathbb{R}^6$$

其中 q_i 为第 i 个旋转关节的角位移（单位：rad），受限于URDF中声明的关节限位 $[q_{i,\min}, q_{i,\max}]$ 。

末端执行器（tool0）在基座坐标系下的位姿由齐次变换矩阵 $\mathbf{T}_{EE}(\mathbf{q}) \in \mathbf{SE}(3)$ 表示：

$$\mathbf{T}_{EE}(\mathbf{q}) = [\mathbf{R}(\mathbf{q}) \ \mathbf{p}(\mathbf{q}); \mathbf{0}^T \ 1]$$

其中 $\mathbf{R}(\mathbf{q}) \in \mathbf{SO}(3)$ 为 3×3 旋转矩阵， $\mathbf{p}(\mathbf{q}) \in \mathbb{R}^3$ 为位置向量。

正运动学由6个关节变换矩阵与末端工具变换的链式乘积给出：

$$\mathbf{T}_{EE}(\mathbf{q}) = \prod_{i=1}^6 (\mathbf{T}_{i^0} \cdot \mathbf{R}(\mathbf{a}_i, q_i)) \cdot \mathbf{T}_{\text{tool}}$$

其中 $\mathbf{T}_{i^0} \in \mathbf{SE}(3)$ 为第 i 个关节的固定原点变换（由URDF中joint origin的xyz/rpy确定）， $\mathbf{a}_i \in \mathbb{R}^3$ 为关节旋转轴单位向量（ $\|\mathbf{a}_i\|=1$ ）， $\mathbf{R}(\mathbf{a}_i, q_i)$ 为绕轴 $\mathbf{a}_i$ 旋转 q_i 的Rodrigues旋转矩阵， $\mathbf{T}_{\text{tool}}$ 为wrist_3坐标系至tool0的固定工具变换。

Rodrigues旋转公式（GPU实现为 `build_rotation_matrix()`）：

$$\mathbf{R}(\mathbf{a}, q) = \mathbf{I} + \sin(q) \cdot [\mathbf{a}]_{\times} + (1 - \cos(q)) \cdot [\mathbf{a}]_{\times}^2$$

其中 $[\mathbf{a}]_{\times}$ 为向量 $\mathbf{a}$ 的反对称矩阵。在GPU上，sin/cos由SFU（特殊功能单元）流水线执行，与算术运算并行。

6.2.2 位姿误差定义

给定目标位姿 $\mathbf{T}^d = [\mathbf{R}^d \ \mathbf{p}^d; \mathbf{0}^T \ 1]$ ，当前位置 $\mathbf{T}(\mathbf{q}) = [\mathbf{R}(\mathbf{q}) \ \mathbf{p}(\mathbf{q}); \mathbf{0}^T \ 1]$ 。

位置误差： $\mathbf{e}_p = \mathbf{p}^d - \mathbf{p}(\mathbf{q}) \in \mathbb{R}^3$

姿态误差：采用李代数形式， $\mathbf{e}_R = \text{Log}(\mathbf{R}^d \cdot \mathbf{R}(\mathbf{q})^T) \mathbf{V} \in \mathbb{R}^3$ 其中Log为SO(3)上的对数映射， $\mathbf{V}$ 为反对称矩阵到向量的逆映射。

总六维误差向量： $\mathbf{e}(\mathbf{q}) = [\mathbf{e}_p; \mathbf{e}_R] \in \mathbb{R}^6$

收敛判据：采用三档递进阈值体系——Loose ($\epsilon_p=0.03 \text{ m} / 30 \text{ mm}$, $\epsilon_R=0.1745 \text{ rad} / 10^\circ$)、Medium ($\epsilon_p=0.01 \text{ m} / 10 \text{ mm}$, $\epsilon_R=0.0873 \text{ rad} / 5^\circ$, 主 benchmark) 和 Strict ($\epsilon_p=0.005 \text{ m} / 5 \text{ mm}$, $\epsilon_R=0.0175 \text{ rad} / 1^\circ$)。位置与姿态误差具有不同物理量纲，因此采用独立阈值进行收敛判定，避免单一加权误差掩盖某一项未收敛的情况。

6.2.3 DLS正规方程

第 k 次迭代中，关节增量 $\Delta \mathbf{q}$ 由如下加权最小二乘问题确定：

$$\Delta \mathbf{q} = \text{argmin}_{\{\Delta \mathbf{q}\}} \|\mathbf{W}(\mathbf{J} \cdot \Delta \mathbf{q} - \mathbf{e})\|_2^2 + \lambda^2 \|\Delta \mathbf{q}\|_2^2$$

其中 $\mathbf{J} \in \mathbb{R}^{6 \times 6}$ 为位姿误差对关节变量的Jacobian， $\mathbf{W}$ 为位置与姿态误差权重矩阵（ 6×6 对角正定）， $\lambda > 0$ 为阻尼系数。

上述优化问题对应的法方程为：

$$(J^T \cdot W^2 \cdot J + \lambda^2 I) \cdot \Delta q = J^T \cdot W^2 \cdot e$$

令阻尼正规矩阵 $H = J^T \cdot W^2 \cdot J + \lambda^2 I \in R^{6 \times 6}$ （Gauss-Newton近似海森），梯度向量 $g = J^T \cdot W^2 \cdot e \in R^6$ ，则每次DLS迭代归结为固定规模线性系统 $H \cdot \Delta q = g$ 。

数值Jacobian计算（中心差分法）：

$$J_{[:,j]} = [e(q+\epsilon \cdot e_j) - e(q-\epsilon \cdot e_j)] / (2\epsilon)$$
，其中 $\epsilon = 1 \times 10^{-6}$ ， e_j 为单位向量。

关节更新与限位投影：

$$q^{(k+1)} = \Pi_Q(q^{(k)} + \alpha \cdot \Delta q)$$

其中 $\alpha \in (0,1]$ 为步长因子， Π_Q 为关节限位投影算子： $\Pi_Q(q_i) = \text{clamp}(q_i, q_{\{i,\text{min}\}}, q_{\{i,\text{max}\}})$ 。

步长 α 由 $\|\Delta q\|_\infty \leq 0.25 \text{ rad}$ 的硬约束确定——若 $\|\Delta q\|_\infty \leq 0.25$ 则 $\alpha=1.0$ （全步长）；否则 $\alpha=0.25/\|\Delta q\|_\infty$ （等比缩放）。

6.3 CUDA并行映射的具体实现

6.3.1 Kernel函数签名与启动配置

主kernel函数 `ik_batch_solve` 的完整签名：

```
__global__ void ik_batch_solve(
    const double* __restrict__ d_targets,      // [N, 16] 目标变换矩阵（行主序）
    const double* __restrict__ d_seeds,        // [N, 6]   初始关节角种子
    double* __restrict__ d_results,            // [N, 6]   输出关节角
    double* __restrict__ d_errors,            // [N, 2]   输出误差(pos_err,
    rot_err)
    double* __restrict__ d_iterations,         // [N]      输出迭代次数
    const int    max_iter,                    // 最大迭代次数 K_max=160
    const double pos_tol,                     // 位置容差
    const double orient_tol,                  // 姿态容差
    const int    N);                          // 批量规模
```

Grid/Block配置：

```
dim3 grid(N, 1, 1);      // N个block，每block一个目标
dim3 block(128, 1, 1);   // 128线程 = 4 Warps × 32 Lanes
ik_batch_solve<<<grid, block>>>(...);
```

6.3.2 共享内存布局

每个Block的共享内存布局（总计约1,616字节）：

变量名	类型	大小	用途
s_q[8]	double	64 B	当前关节角（6个有效+2个padding）

变量名	类型	大小	用途
s_T[16]	double	128 B	当前FK结果（4×4齐次矩阵）
s_T_tgt[16]	double	128 B	目标位姿（4×4齐次矩阵）
s_J[SMEM_J_SIZE]	double	384 B (padded) / 288 B (raw)	Jacobian矩阵（PaddedMat6x8）
s_H[SMEM_H_SIZE]	double	384 B (padded) / 288 B (raw)	阻尼正规矩阵（PaddedMat6x8）
s_err[6]	double	48 B	位姿误差向量
s_g[6]	double	48 B	梯度向量
s_dq[6]	double	48 B	关节增量
s_q_ref[6]	double	48 B	参考关节角（种子）
s_q_best[6]	double	48 B	历史最优关节角（fallback）
s_converged	int	4 B	收敛标志
s_iter_count	int	4 B	迭代计数器
s_lambda	double	8 B	阻尼系数
s_best_pos_err	double	8 B	历史最优位置误差
s_stagnation	int	4 B	停滞计数器

其中SMEM_J_SIZE和SMEM_H_SIZE由PADDEDMAT_STRIDE决定（消融级别≥B2时PADDEDMAT_STRIDE=8，对应48个double/矩阵；否则PADDEDMAT_STRIDE=6，对应36个double/矩阵）。

6.3.3 常量内存中的运动学参数

UR10运动学参数（总计1,384字节）存储在__constant__内存中，通过64 KB constant cache广播至所有block：

常量数组	元素数	类型	大小	内容
c_segment_origins	96	double	768 B	6段关节原点变换矩阵（每段16个元素）
c_segment_axes	18	double	144 B	6×3维旋转轴向量
c_q_index	6	int	24 B	关节索引映射
c_T_wrist3_to_tcp	16	double	128 B	工具坐标系变换
c_lambda_params	4	double	32 B	自适应阻尼参数
c_joint_limits	12	double	96 B	关节限位 [min, max] × 6
c_weight_schedule	6	double	48 B	位姿误差权重

6.3.4 DLS迭代主循环的伪代码

以下为每个Block内部执行的DLS迭代主循环（K_max=160次迭代）：


```

// Phase 1: 初始化
从全局内存加载目标位姿 → s_T_tgt
从全局内存加载初始种子 → s_q, s_q_ref, s_q_best
s_lambda ← λ_base (2e-4)
s_stagnation ← 0
s_best_pos_err ← INFINITY

// Phase 2: DLS迭代循环
for iter = 0 to max_iter-1:

    // 2a. 前向运动学 (Warp 0, Lane 0)
    if lane_id == 0:
        forward_kinematics(s_q, s_T) // 读取 __constant__ 内存
        __syncwarp()

    // 2b. 位姿误差计算 (Warp 0, Lane 0)
    if lane_id == 0:
        s_err = pose_error(s_T, s_T_tgt)
        s_pos_err = ||s_err[0:3]||2
        rot_geodesic = rotation_geodesic_distance(s_T, s_T_tgt)
        __syncwarp()

    // 2c. 收敛判定 (Warp 0, Lane 0)
    if lane_id == 0:
        if s_pos_err <= pos_tol AND rot_geodesic <= orient_tol:
            s_converged = 1
            break
        if s_pos_err < s_best_pos_err:
            s_best_pos_err = s_pos_err
            s_q_best = s_q
            s_stagnation = 0
        else:
            s_stagnation += 1
    __syncthreads()
    if s_converged: break

    // 2d. 数值Jacobian (Warp 1, 6路数据并行)
    if lane_id >= 32 AND lane_id < 38: // Lanes 32-37
        j = lane_id - 32
        q_plus = s_q; q_plus[j] += ε
        forward_kinematics(q_plus, T_plus)
        q_minus = s_q; q_minus[j] -= ε
        forward_kinematics(q_minus, T_minus)
        J(0:2, j) = (T_plus.pos - T_minus.pos) / (2ε)
        J(3:5, j) = rotation_column_diff(T_plus, T_minus) / (2ε)
    __syncthreads()

    // 2e. 自适应阻尼 (Warp 0, Lane 0)
    if lane_id == 0:
        if iter == 0:
            // 距离初始化
            if s_pos_err > 0.1:

```

```

        s_lambda = max( $\lambda_{\text{base}}$ ,  $\lambda_{\text{far}} * s_{\text{pos\_err}} / \lambda_{\text{scale}}$ )
    else:
        s_lambda =  $\lambda_{\text{floor}}$  +  $\lambda_{\text{base}} * s_{\text{pos\_err}} / \lambda_{\text{scale}}$ 
    else:
        // Marquardt误差驱动更新
        if s_pos_err < s_prev_pos_err:
            s_lambda *= 0.7 // 改进→减小阻尼→趋向Gauss-Newton
        else:
            s_lambda *= 2.0 // 退化→增大阻尼→趋向梯度下降
    s_lambda = clamp(s_lambda, 1e-4, 0.5)
    // 停滞超驰
    if s_stagnation > 12:
        s_lambda = min(s_lambda * 5.0, 0.5)
__syncthreads()

// 2f. 阻尼正规矩阵H构造 (Warp 2, 36路数据并行)
if lane_id >= 64 AND lane_id < 100: // Lanes 64-99
    row = (lane_id - 64) / 6
    col = (lane_id - 64) % 6
    sum = 0.0
    for k = 0 to 5:
        w_k = c_weight_schedule[k]
        sum += J(k, row) * w_k * w_k * J(k, col)
    if row == col: sum += s_lambda
    H(row, col) = sum
__syncthreads()

// 2g. 梯度向量g构造 (Warp 3的部分Lane, 6路数据并行)
if lane_id >= 96 AND lane_id < 102: // Lanes 96-101
    i = lane_id - 96
    sum = 0.0
    for k = 0 to 5:
        w_k = c_weight_schedule[k]
        sum += J(k, i) * w_k * w_k * s_err[k]
    s_g[i] = sum
__syncthreads()

// 2h. LDLT求解 (Warp 0, Lane 0)
if lane_id == 0:
    copy H(PaddedMat6x8) → H_dense[36] (寄存器)
    copy s_g[6] → g_dense[6] (寄存器)
    ldlt_solve_6x6(H_dense, g_dense, s_dq) // 86次标量运算, 全寄存器
__syncthreads()

// 2i. 关节更新与限位投影 (Warp 3, 6路数据并行)
if lane_id >= 96 AND lane_id < 102:
    i = lane_id - 96
    dq_infnorm = max(|s_dq[0]|, ..., |s_dq[5]|)
    if dq_infnorm < 1e-8:
        s_converged = 1 // 停滞收敛
        break
    if dq_infnorm > 0.25:
        scale = 0.25 / dq_infnorm
        s_dq[i] *= scale

```

```

        q_new = s_q[i] + s_dq[i]
        s_q[i] = clamp(q_new, c_joint_limits[2*i], c_joint_limits[2*i+1])
    __syncthreads()
    s_prev_pos_err = s_pos_err

// Phase 3: 写回结果
if lane_id == 0:
    d_results[blockIdx.x * 6 : (blockIdx.x+1) * 6] = s_q (或 best_s_q)
    d_errors[blockIdx.x * 2] = s_best_pos_err
    d_errors[blockIdx.x * 2 + 1] = rotation_geodesic
    d_iterations[blockIdx.x] = s_iter_count

```

6.3.5 自适应阻尼策略的实现

自适应阻尼策略（B3及以上消融级别启用）实现了LM混合阻尼调度，包含三层逻辑：

层一：距离初始化（迭代0）

```

if pos_err > 0.1:
    λ = max(λ_base, λ_far × pos_err / λ_scale)
    λ = min(λ, λ_far × 3.0)
else:
    λ = λ_floor + λ_base × pos_err / λ_scale

```

层二：Marquardt误差驱动更新（迭代1+）

```

if pos_err < prev_pos_err: λ *= 0.7 // 改进
else: λ *= 2.0 // 退化
λ = clamp(λ, 1e-4, 0.5)

```

层三：停滞超驰

```

if stagnation > 12:
    λ = min(λ × 5.0, 0.5)

```

其中 $\lambda_{\text{base}}=2\text{e-}4$, $\lambda_{\text{far}}=5\text{e-}2$, $\lambda_{\text{floor}}=1\text{e-}4$, $\lambda_{\text{scale}}=8\text{e-}2$ ，存储于 `__constant__` 内存 `c_lambda_params[4]` 中。

该策略的关键特征是：分段线性、距离比例初始化、误差驱动更新——相对于完整LM的平方根/指数运算，额外计算开销极小（仅含if-else、乘法、fmin/fmax），适合GPU SIMT执行模型。实测效果：将收敛率从43-83%（固定 λ ，Medium阈值）恢复至99.8-100%，迭代次数减少59-83%。

6.4 LDLT寄存器求解器的具体实现

6.4.1 函数接口

```
__device__ __forceinline__ void ldlt_solve_6x6(
    const double* H,    // [in]  6×6对称正定矩阵（行主序，36个元素）
    const double* g,    // [in]  6维梯度向量
    double* dq);        // [out] 6维解向量Δq
```

`__device__ __forceinline__` 双重属性确保编译器将该函数内联至调用点，所有局部变量分配在寄存器中。

6.4.2 完整算法实现

步骤1：拷贝输入至局部数组

```
double A[6][6]; // 寄存器内6×6矩阵
for (i = 0..5, j = 0..5):
    A[i][j] = H[i*6 + j]
```

步骤2：LDLT分解

```
double L[6][6] = {0}; // 单位下三角矩阵（寄存器）
double D[6];          // 对角矩阵（寄存器）

for j = 0 to 5:
    // 对角元:  $D[j] = A[j][j] - \sum_{k<j} L[j][k]^2 \cdot D[k]$ 
    D[j] = A[j][j]
    for k = 0 to j-1:
        D[j] -= L[j][k] * L[j][k] * D[k]

    L[j][j] = 1.0

    // 非对角元:  $L[i][j] = (A[i][j] - \sum_{k<j} L[i][k] \cdot L[j][k] \cdot D[k]) / D[j]$ 
    for i = j+1 to 5:
        L[i][j] = A[i][j]
        for k = 0 to j-1:
            L[i][j] -= L[i][k] * L[j][k] * D[k]
        L[i][j] /= D[j]
```

步骤3：前代求解 $L \cdot y = g$

```
double y[6];
for i = 0 to 5:
    y[i] = g[i]
    for k = 0 to i-1:
        y[i] -= L[i][k] * y[k]
```

步骤4：对角缩放 $D \cdot z = y$

```
for i = 0 to 5:
    y[i] /= D[i]
```

步骤5：回代求解 $L^T \cdot \Delta q = z$

```
for i = 5 downto 0:
    dq[i] = y[i]
    for j = i+1 to 5:
        dq[i] -= L[j][i] * dq[j]
```

所有循环在编译时完全展开——三重嵌套的迭代次数（ $j=0..5, k=0..j-1, i=j+1..5$ ）为编译时常量，NVCC `#pragma unroll` 将其展开为约86条标量运算的直线代码序列。该求解器的三个核心属性——FP64精度（15位有效数字）、固定成本（每轮~0.1 μ s，波动<0.3%）和零内存流量——共同构成了本发明三档阈值鲁棒性的结构基础。

6.5 PaddedMat6x8的具体实现**6.5.1 C++封装代码**

```
struct PaddedMat6x8 {
    double* data;

    __device__ __forceinline__ PaddedMat6x8(double* d) : data(d) {}

    __device__ __forceinline__ double& operator()(int row, int col) {
        return data[row * PADDEDMAT_STRIDE + col];
    }

    __device__ __forceinline__ const double& operator()(int row, int col)
const {
        return data[row * PADDEDMAT_STRIDE + col];
    }
};

// FP32变体（用于混合精度B5）
struct FloatPaddedMat6x8 {
    float* data;

    __device__ __forceinline__ FloatPaddedMat6x8(float* d) : data(d) {}

    __device__ __forceinline__ float& operator()(int row, int col) {
        return data[row * PADDEDMAT_STRIDE + col];
    }
};
```

6.5.2 Bank冲突消除的数学原理

对于stride=8（PADDEDMAT_STRIDE=8）的布局：

每行占8个double = 64 bytes = 16个Bank（每个Bank 4 bytes）。

- 偶数行（r=0,2,4）：Bank起始于 $c \times 2 \bmod 32$ ，使用Bank 0-15
- 奇数行（r=1,3,5）：Bank起始于 $16 + c \times 2 \bmod 32$ ，使用Bank 16-31

因为 $\gcd(16,32)=16$ ，偶/奇数行使用的Bank集合完全不重叠。在核心计算中（每次 ≤ 2 行同时访问），Bank冲突被系统性消除。

6.6 混合精度的具体实现

6.6.1 精度边界的关键代码

FP32 → FP64转换（LDLT^T求解入口）：

```
// 将FP32共享内存中的H和g转换为FP64用于LDLTT求解
double H_dense[36];
double g_dense[6];
for (int r = 0; r < 6; r++) {
    for (int c = 0; c < 6; c++) {
        H_dense[r * 6 + c] = (double)H(r, c); // FP32 → FP64
    }
    g_dense[r] = (double)s_g[r]; // FP32 → FP64
}
ldlt_solve_6x6(H_dense, g_dense, s_dq); // 全部FP64运算，86次标量操作
```

FP64 → FP32写回（关节更新入口）：

```
// s_dq为FP64，关节更新使用FP32
float dq_f[6];
for (int i = 0; i < 6; i++) {
    dq_f[i] = (float)s_dq[i]; // FP64 → FP32
    s_q[i] = clamp(s_q[i] + dq_f[i], lo[i], hi[i]);
}
```

6.6.2 混合精度kernel的Nsight Compute性能画像

B5配置在N=100条件下的关键NCU指标：

指标	值	解读
Compute throughput	60.73%	计算为主瓶颈
DRAM throughput	1.16%	非带宽受限
Registers/thread	98	略高于B4（FP32 → FP64转换逻辑）

指标	值	解读
Achieved occupancy	33.3%	受寄存器上限约束
L1/TEX hit rate	98.62%	数据复用良好
Shared bank conflicts	1,295	较B4减少49%
Kernel duration	827 μ s	较B4 2,920 μ s缩短72%

Kernel执行时间从B4的2,920 μ s降至827 μ s（缩短72%），该加速来自FP32计算延迟降低、共享内存带宽利用率提升以及bank conflict减少三者的叠加效应，其中计算延迟降低贡献最大。

6.7 消融编译控制系统

6.7.1 消融级别定义

本发明包含两类设计选择：**结构性设计项**（框架核心架构特征，所有消融级别默认启用）与**可量化消融项**（可在编译时独立切换，其性能影响可直接测量）。

结构性设计项包括：（1）1 block/target、128 threads/block、四 warp 分工；（2）寄存器内 6×6 LDLT 小矩阵求解；（3）单 kernel 内完成全部 DLS 迭代。

可量化消融项的逐级配置如表所示：

级别	ConstMem	PaddedMat6x8	Adaptive Damping	Step Clamp	Branch Align	Precision
B0	—	—	—	—	—	FP64
B1	✓	—	—	—	—	FP64
B2	✓	✓	—	—	—	FP64
B3	✓	✓	✓	—	—	FP64
B4	✓	✓	✓	✓	✓	FP64
B5	✓	✓	✓	—	—	FP32+FP64
B6	✓	✓	✓	—	—	FP32+FP64+ Graph

B0为FP64无优化基线，B1-B2追踪内存层次优化的逐步贡献，B3引入自适应阻尼（所有消融变更中对收敛率影响最大的单一因素），B5在B3的基础上切换为混合精度，B6在B5基础上叠加CUDA Graph。

6.7.2 预处理器条件编译

```
#ifndef ABLATION_LEVEL
#define ABLATION_LEVEL 5 // 默认：B5混合精度
#endif

// PaddedMat6x8 stride控制
#if ABLATION_LEVEL < 2
```

```
#define PADDEDMAT_STRIDE 6    // B0/B1: 非填充
#else
#define PADDEDMAT_STRIDE 8    // B2+: 填充
#endif

// 常量内存 vs 全局内存
#if ABLATION_LEVEL >= 1
    // 使用 __constant__ 内存 (B1+)
    forward_kinematics(s_q, s_T); // 内部读取c_segment_origins等
#else
    // 使用全局内存参数 (B0)
    forward_kinematics_global(s_q, s_T, d_origins, d_axes, d_q_index,
d_T_tcp);
#endif

// 自适应阻尼 (B3+)
#if ABLATION_LEVEL >= 3
    // 距离初始化 + Marquardt误差驱动更新 + 停滞超驰
#endif

// 步长钳位 + 分支对齐 (仅B4启用)
#if ABLATION_LEVEL == 4
    // 0.35 rad步长裁剪 + atan2分支对齐
#endif

// 混合精度 (B5/B6启用)
#if ABLATION_LEVEL >= 5
    // FP32 FK/Jacobian/Hessian + FP64 LDLT
#endif
```

6.8 7DOF扩展实施例

从UR10（6DOF）到Panda（7DOF）的迁移仅需以下维度常数替换：

参数	UR10 (6DOF)	Panda (7DOF)
DOF	6	7
Jacobian规模	6×6	6×7
阻尼正规矩阵H规模	6×6	7×7
LDLT规模	6×6	7×7
FK链段数	6	7
共享内存J大小	6×PADDEDMAT_STRIDE	7×PADDEDMAT_STRIDE
共享内存H大小	6×PADDEDMAT_STRIDE	7×PADDEDMAT_STRIDE

Warp分工方案保持不变（线程映射、warp角色分配、自适应阻尼控制逻辑均不变）。正确性验证结果：CUDA FK与CPU参考在机器精度量级一致（最大逐元素误差 2.78×10^{-16} ），IK收敛率（50% at N=10）与Python CPU参考完全一致，验证核心数值路径未被破坏。

6.9 批量扩展确定性的工程意义

本发明的一项关键工程特性是批量扩展的完全确定性。在N=100→10000全量程扫描（12个N值，步长1000，Medium阈值10mm/5°）中：

N	CUDA B5 Throughput	GPU时间(ms)	收敛率
100	112,414	0.89	1.000
500	158,251	3.16	0.998
1000	148,412	6.74	0.998
2000	156,007	12.82	1.000
3000	160,785	18.66	1.000
4000	166,898	23.97	0.999
5000	168,683	29.64	0.9998
6000	167,030	35.92	0.9998
7000	166,933	41.93	0.9996
8000	173,709	46.05	0.9999
9000	164,979	54.55	0.9999
10000	165,550	60.40	0.9996

吞吐保持在148k-174k窄区间（±8%），GPU时间与N严格线性（R² > 0.999）。该确定性来源于三项设计决策：（1）每目标固定2KB中间状态，裸cudaMalloc一次性分配，不存在caching allocator碎片化问题；（2）无PyTorch依赖——不经过任何第三方内存管理器；（3）单kernel全迭代封装——kernel launch后GPU完全自治，host侧在kernel执行期间不进行任何内存操作。

七、权利要求建议

以下为供专利代理人参考的权利要求框架，最终权利要求的措辞和范围应由执业专利代理人根据相关法律法规进行调整。

独立权利要求1（方法）

一种面向标准机械臂批量逆运动学的CUDA并行映射与寄存器级小矩阵求解方法，其特征在于，包括以下步骤：

步骤S1：将N个目标末端位姿中的每一个映射为1个CUDA block，各block之间独立并行求解，Grid维度定义为(N, 1, 1)；

步骤S2：每个block包含128个线程（4个Warp×32个Lane），block内按Warp分工协作：Warp 0负责前向运动学计算、位姿误差计算、LDLT线性系统求解、自适应阻尼更新和收敛判定；Warp 1负责6列数值雅可比矩阵的并行组装；Warp 2负责6×6阻尼正规矩阵和梯度向量的并行构造；Warp 3负责关节角步长更新和限位投影；

步骤S3：在单次kernel launch中执行全部N个目标的全部DLS迭代（单目标至多K_max=160次迭代），各block独立运行至收敛或达到最大迭代次数，无需CPU端中途干预和重新launch；

步骤S4：每次DLS迭代中的 6×6 阻尼正规矩阵线性系统 $H \cdot \Delta q = g$ 的求解采用寄存器驻留的LDLT分解方法——将H矩阵、L矩阵、D对角元和中间向量的全部元素置于GPU寄存器中，无需全局内存和共享内存访问，全部循环在编译时展开为直线代码序列，总计约86次标量运算（65次可映射为FMA指令）；

步骤S5：在共享内存中， 6×6 矩阵J和H采用PaddedMat6x8布局——行步长从自然值6扩展为8，添加2列零填充，使偶数行和奇数行使用不相重叠的Bank集合（偶数行占用Bank 0-15，奇数行占用Bank 16-31），消除stride=6布局导致的系统性Bank冲突；

步骤S6：采用FP32/FP64混合精度计算路径——前向运动学、位姿误差计算和数值雅可比组装在FP32精度下执行（约90%计算量）；阻尼正规矩阵H和梯度向量g的元素以FP64精度累积；LDLT分解求解、自适应阻尼更新和收敛判定在FP64精度下执行；关节更新以FP32精度执行。

从属权利要求

权利要求2（根据权利要求1所述的方法，针对Block/Warp映射）：所述步骤S2中，Grid维度定义为(N,1,1)，Block维度定义为(128,1,1)，任务标识 $b = \text{blockIdx.x}$ ；Warp编号 $w = \lfloor \text{threadIdx.x} / 32 \rfloor$ ，Lane编号 $l = \text{threadIdx.x} \bmod 32$ ；Warp 0的Lane 0执行全部控制流逻辑（FK、误差计算、LDLT求解、阻尼更新、收敛判定），Warp 1的Lane $32+j$ ($j=0,\dots,5$) 负责Jacobian第j列的6路数据并行组装，Warp 2的Lane $64+(i \times 6+j)$ ($i,j=0,\dots,5$) 负责 $H_{\{ij\}}$ 元素的36路数据并行构造，Warp 3的Lane $96+i$ ($i=0,\dots,5$) 负责第i个关节的6路数据并行更新。

权利要求3（根据权利要求1所述的方法，针对LDLT求解器）：所述步骤S4中，LDLT分解求解包含四个阶段：阶段一为LDLT分解（对角元更新15次乘加、非对角元更新20次乘加、L缩放15次除法）；阶段二为前代求解 $L \cdot y = g$ （15次乘减）；阶段三为对角缩放 $D \cdot z = y$ （6次除法）；阶段四为回代求解 $L^T \cdot \Delta q = z$ （15次乘减）；所有中间变量（H矩阵36个FP64元素、L矩阵36个FP64元素、D对角元6个FP64元素、中间向量18个FP64元素，合计768字节）完全驻留在单个Lane的寄存器文件中，不产生全局内存流量和local memory spilling。

权利要求4（根据权利要求1所述的方法，针对PaddedMat6x8）：所述步骤S5中，PaddedMat6x8行地址映射为 $\text{addr}(r,c) = r \times \text{PADDED_MAT_STRIDE} + c$ ，其中PADDED_MAT_STRIDE=8；所述零填充列（列索引6和7）的值为0；Bank映射公式为 $B(r,c) = (r \times 16 + c \times 2) \bmod 32$ ；由于行步长16与Bank总数32的最大公约数 $\text{gcd}(16,32)=16$ （而自然布局的 $\text{gcd}(12,32)=4$ ），偶数行和奇数行使用的Bank集合完全不重叠，在核心计算中（每次 ≤ 2 行同时访问）Bank冲突被系统性消除。

权利要求5（根据权利要求1所述的方法，针对混合精度转换边界）：所述步骤S6中，FP32 $\rightarrow$ FP64类型转换发生在LDLT求解入口处——将FP32共享内存中的Hessian矩阵和梯度向量显式转换为FP64寄存器数组后调用全FP64 `ldlt_solve_6x6()`函数；FP64 $\rightarrow$ FP32类型转换发生在关节更新处——将FP64求解结果 Δq 转换为FP32后进行关节限位投影；中间不存在其他隐式类型转换或混合精度运算。

权利要求6（根据权利要求1所述的方法，针对自适应阻尼策略）：所述步骤S2的Warp 0执行的自适应阻尼策略包括三层逻辑：（层一）迭代0时基于目标距离的距离初始化——远目标（ $\|e_p\|_2 > 0.1m$ ） $\lambda = \max(\lambda_{\text{base}}, \lambda_{\text{far}} \times \|e_p\|_2 / \lambda_{\text{scale}})$ ，近目标 $\lambda = \lambda_{\text{floor}} + \lambda_{\text{base}} \times \|e_p\|_2 / \lambda_{\text{scale}}$ ；（层二）迭代1+时基于误差改进/退化的分段线性Marquardt更新——位置误差改进时 $\lambda^* = 0.7$ ，退化时 $\lambda^* = 2.0$ ， $\lambda \in [1e-4, 0.5]$ ；（层三）连续停滞超过12次迭代时的超驰机制 $\lambda^* = 5.0$ 。

权利要求7（根据权利要求1所述的方法，针对消融编译控制系统）：通过ABLATION_LEVEL预处理器宏和条件编译指令（`#if ABLATION_LEVEL >= N`），从单一代码库编译生成B0-B6七级消融可执行文件；B0为FP64无优化基线，B1在B0基础上启用常量内存（**constant**），B2在B1基础上启用PaddedMat6x8共享内存布局（PADDED_MAT_STRIDE=8），B3在B2基础上启用自适应阻尼，B4在B3基础上启用步长钳位和分支对齐，B5在B3基础上启用FP32/FP64混合精度，B6在B5基础上启用CUDA Graph replay。

权利要求8（根据权利要求1所述的方法，针对7DOF和参数化扩展）：所述方法通过编译时常量DOF定义自由度维度——将DOF从6替换为7、Jacobian规模从 6×6 扩展为 6×7 、阻尼正规矩阵H规模从 6×6 扩展为 7×7 、LDLT求解规模从 6×6 扩展为 7×7 、FK链段数从6增至7，即可迁移至7自由度串联机械臂的批量IK求解，且Warp分工方案、线程映射、PaddedMat 6×8 布局 and 自适应阻尼控制逻辑保持不变。

权利要求9（根据权利要求1所述的方法，针对单kernel结构的性能确定性）：所述步骤S3中，全部N个目标的全部DLS迭代在单次kernel launch中完成，kernel launch后GPU完全自治，host侧在kernel执行期间不进行任何内存操作、stream同步或设备-主机通信；每目标固定分配约2KB中间状态（裸cudaMalloc一次性分配），不依赖任何第三方内存管理器（如PyTorch CUDA Caching Allocator）；由此保证GPU时间与N严格线性（实测 $R^2 > 0.999$ ），吞吐量在任意批量规模下均可从事先的计算精确预测（实测 $N=100 \rightarrow 10000$ 范围内吞吐波动仅 $\pm 8\%$ ）。

独立权利要求10（系统/装置）

一种面向标准机械臂批量逆运动学的CUDA并行求解系统，其特征在于，包括：

- （a）URDF模型解析模块，用于从标准URDF格式的机器人描述文件中提取关节运动学参数，包括关节原点变换矩阵、旋转轴向量、关节索引映射、工具坐标系变换和关节限位；
- （b）CUDA常量参数导出模块，用于将解析后的运动学参数导出为GPU常量内存（`__constant__`）可加载的C++头文件，总计1,384字节运动学参数通过64 KB constant cache广播至所有block；
- （c）GPU求解器模块，包含实现权利要求1-9任一项所述方法的CUDA kernel函数，以及PaddedMat 6×8 /FloatPaddedMat 6×8 共享内存封装、寄存器级ldlt_solve_6x6()求解器和混合精度类型转换辅助函数；
- （d）消融编译系统，通过ABLATION_LEVEL预处理器宏和条件编译指令从单一代码库生成B0-B6七级消融可执行文件，用于量化各优化项的独立贡献；
- （e）Benchmark评测模块，用于统一调用各求解器（CUDA/cuRobo/PyRoki/KDL/numeric_dls）、采集CUDA event时间指标和收敛率、执行标准化实验规程（3次预热+30次正式重复，算术均值汇总）、输出可复现的CSV/JSON评测结果。

联系人：[填写发明人姓名] **联系电话：**[填写联系电话] **电子邮箱：**[填写电子邮箱] **日期：**2026年6月12日